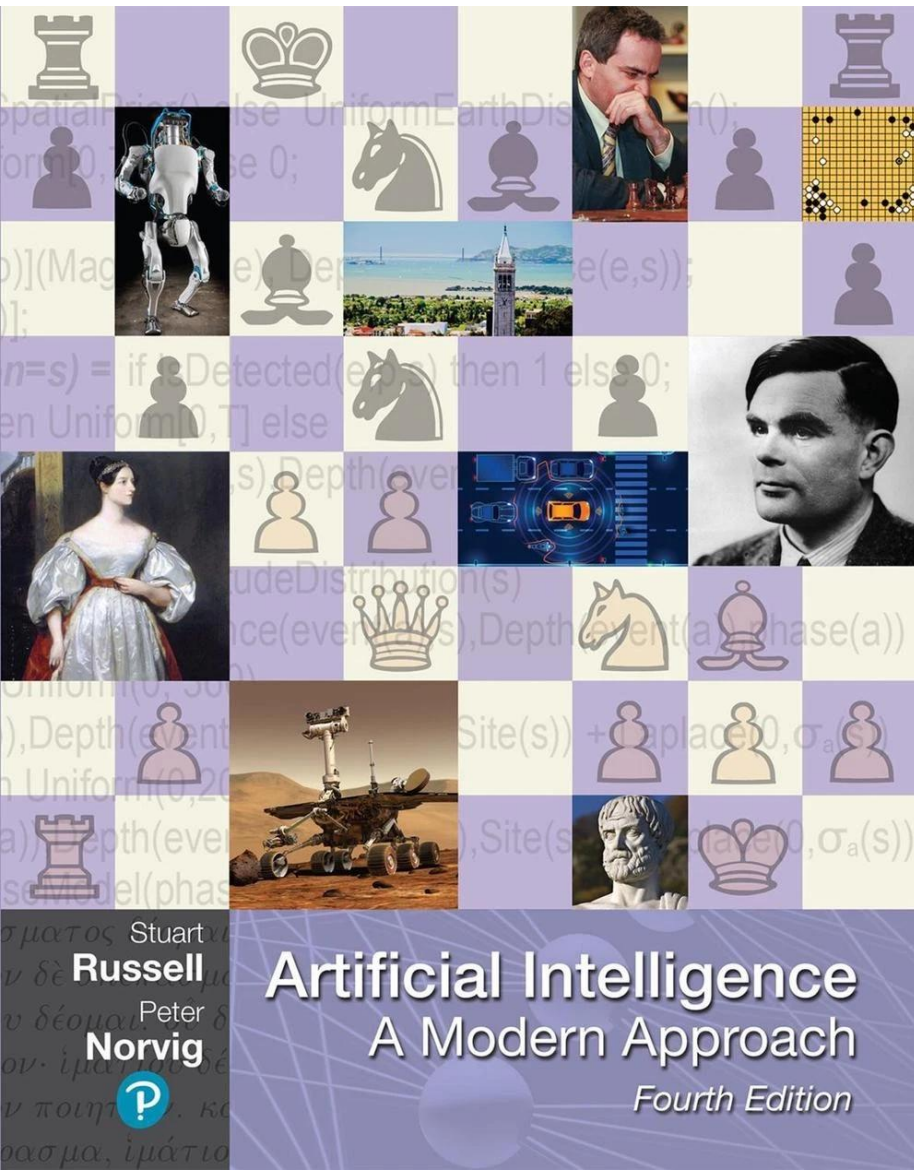
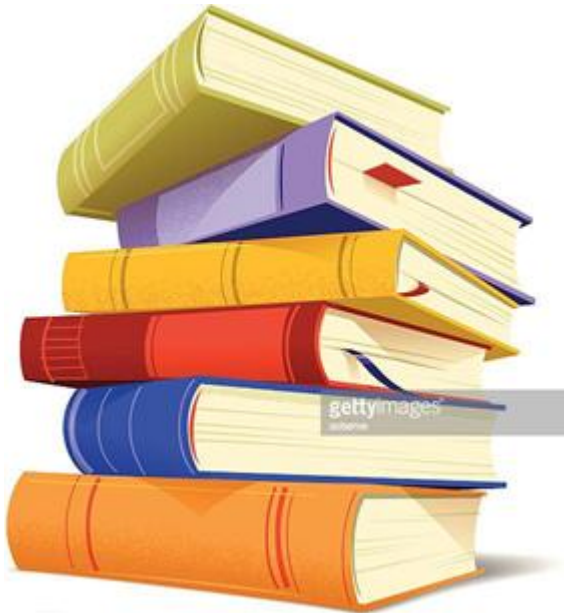


Chapter 24

Deep Learning for Natural Language Processing



Contents



- Word Embeddings
- Recurrent Neural Networks for NLP
- Sequence-to-Sequence Models
- The Transformer Architecture
- Pretraining and Transfer Learning
- State of the art

Word Embeddings



Why Deep Learning Changed NLP

- Language has **structure**, but also endless variation.
- Neural models learn representations directly from data.
- Dense vectors support generalization across related words and contexts.
- Chapter 24 extends Chapter 23 from symbolic models to learned representations.

From One-Hot Symbols to Embeddings

- **One-hot vectors** identify words but do not encode similarity.
- **Word embeddings** map words to dense, low-dimensional vectors.
- Related words tend to occupy nearby regions in vector space.
- Embeddings are learned automatically from text or task supervision.

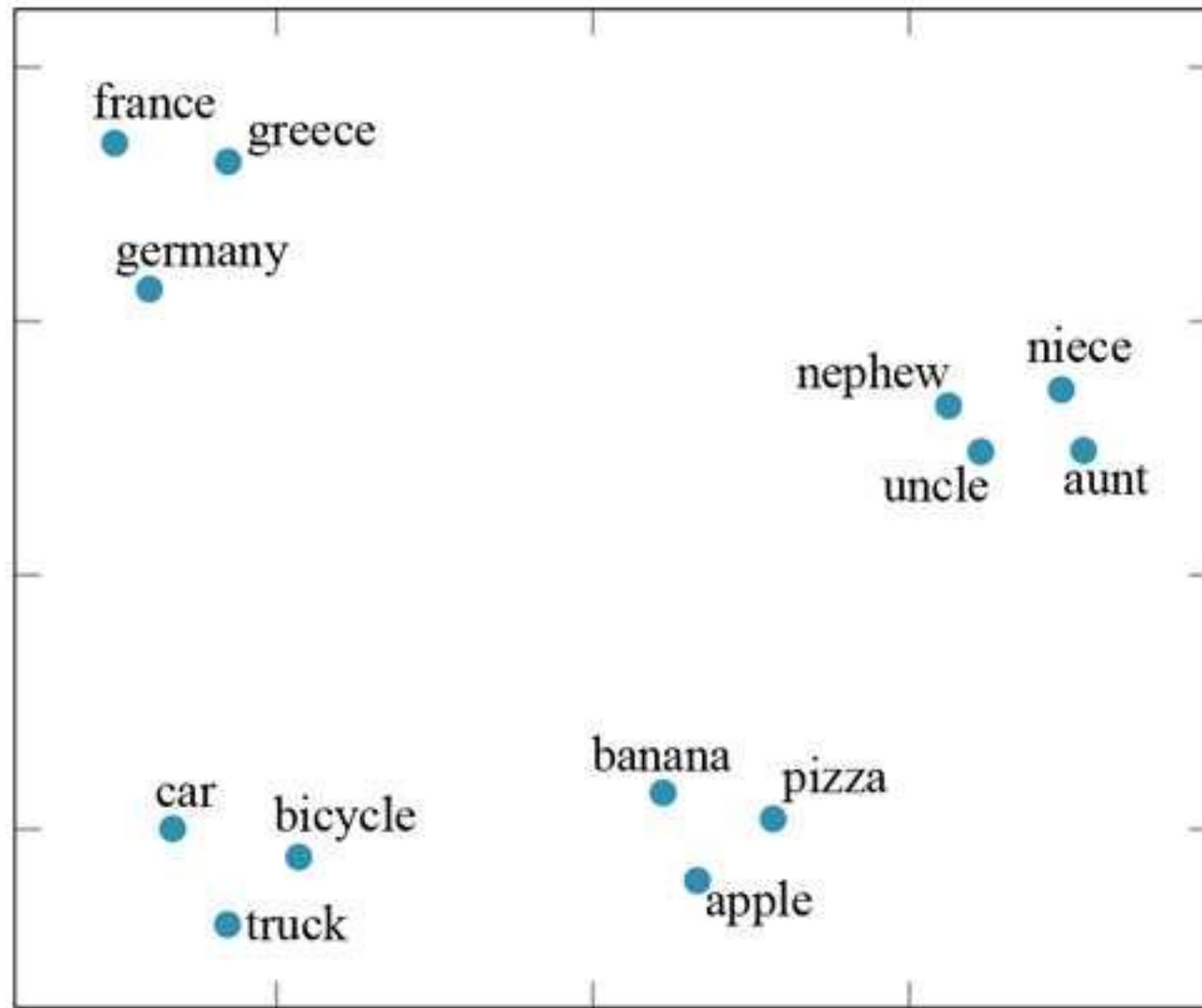


Figure 24.1. GloVe word embeddings projected into two dimensions.

What Embedding Dimensions Mean

- Individual dimensions usually lack clear human-readable meanings.
- The **geometry** of the whole vector space is meaningful.
- Similarity is often measured by distance or cosine similarity.
- Embeddings support downstream tasks more than direct interpretation.

Embedding Geometry and Analogies

- Vector offsets can capture regularities: **king – man + woman $\approx$ queen.**
- Analogies are solved by searching for the nearest vector.
- The effect is useful but not guaranteed for every relation.
- Quality depends on corpus, training objective, and vocabulary coverage.

A	B	C	D = C + (B - A)	Relationship
Athens	Greece	Oslo	Norway	<i>Capital</i>
Astana	Kazakhstan	Harare	Zimbabwe	<i>Capital</i>
Angola	kwanza	Iran	rial	<i>Currency</i>
copper	Cu	gold	Au	<i>Atomic Symbol</i>
Microsoft	Windows	Google	Android	<i>Operating System</i>
New York	New York Times	Baltimore	Baltimore Sun	<i>Newspaper</i>
Berlusconi	Silvio	Obama	Barack	<i>First name</i>
Switzerland	Swiss	Cambodia	Cambodian	<i>Nationality</i>
Einstein	scientist	Picasso	painter	<i>Occupation</i>
brother	sister	grandson	granddaughter	<i>Family Relation</i>
Chicago	Illinois	Stockton	California	<i>State</i>
possibly	impossibly	ethical	unethical	<i>Negative</i>
mouse	mice	dollar	dollars	<i>Plural</i>
easy	easiest	lucky	luckiest	<i>Superlative</i>
walking	walked	swimming	swam	<i>Past tense</i>

Figure 24.2. Word analogies solved with embedding-vector arithmetic.

Supervised Embeddings for POS Tagging

- A neural tagger can learn embeddings while learning the task.
- A fixed-size **word window** provides local context.
- The middle word is classified using surrounding embeddings.
- Embedding weights and network weights are trained together.

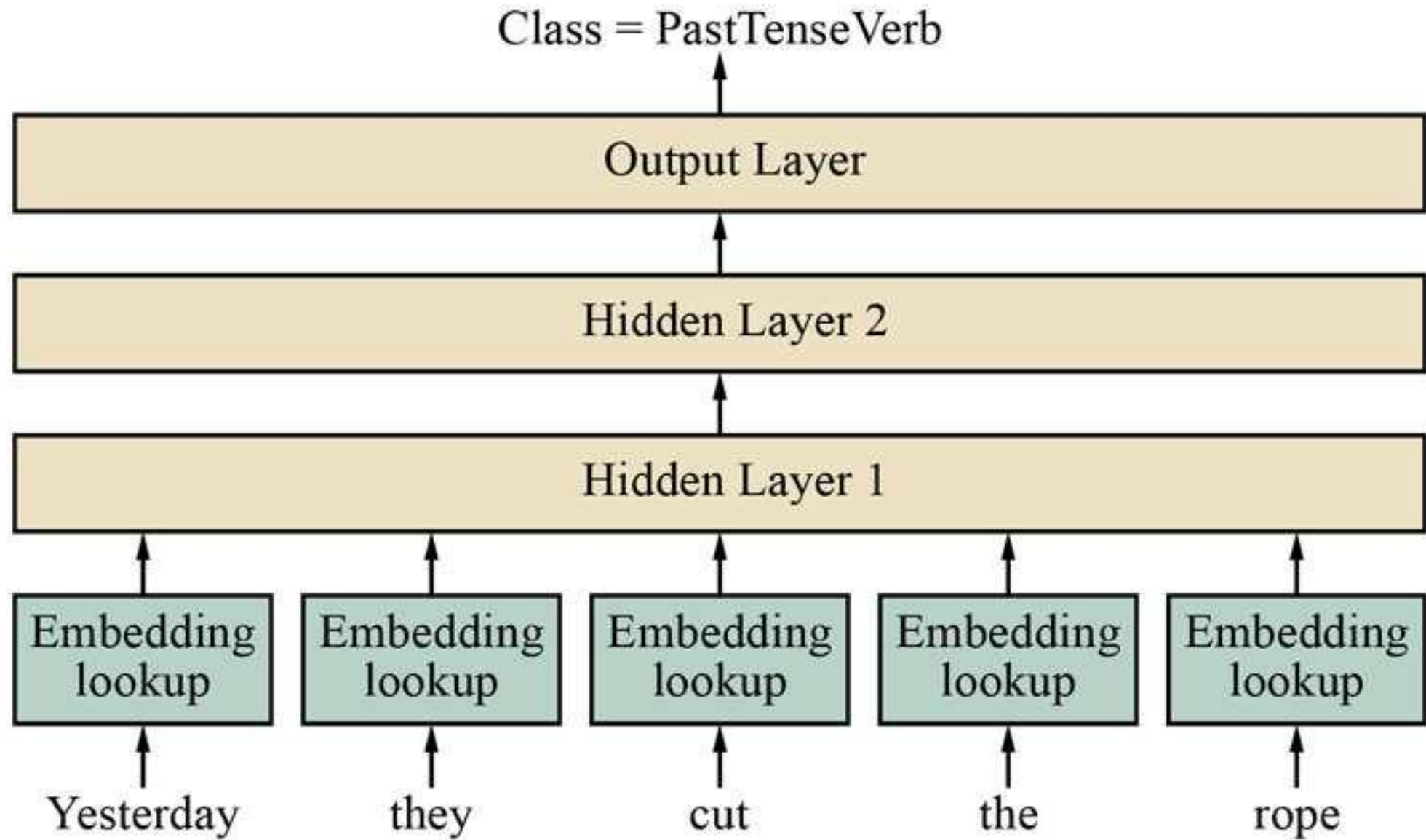


Figure 24.3. Feedforward POS tagging model using a five-word window.

Character-Level Models

- Characters can be encoded as one-hot vectors too.
- Character models learn morphology from prefixes, suffixes, and spelling.
- They handle rare or unseen words better than fixed word vocabularies.
- Most NLP systems still use word or subword-level representations.

Limits of Static Word Embeddings

- A word receives one vector regardless of context.
- Polysemy is hard: **rose** can be a flower or a verb form.
- Idioms may not equal the sum of their component words.
- Biases in training corpora can be reflected in embeddings.

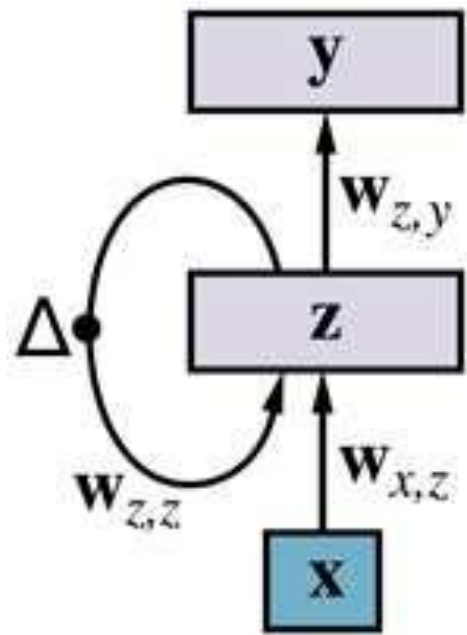


Why RNNs for Language?

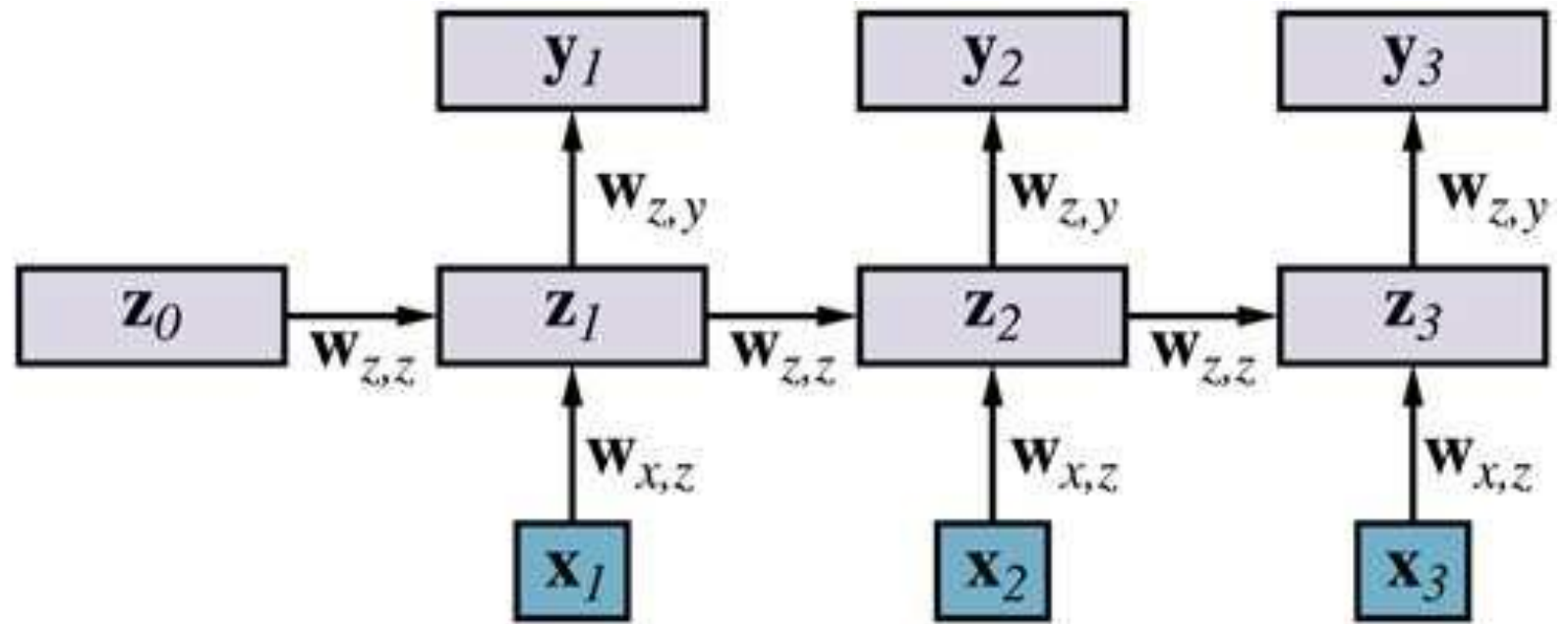
- Language is an **ordered sequence**, not a fixed bag of words.
- RNNs carry a hidden state from one timestep to the next.
- The hidden state can store relevant prior context.
- The same weights are reused across sequence positions.

RNN Language Models

- Input at timestep t is the embedding of word w_t .
- The hidden state summarizes previous words.
- The output is a softmax over the vocabulary for w_{t+1} .
- Training uses unlabeled text: the next word is the label.



(a)



(b)

Figure 24.4. A recurrent neural network and its unrolled computation.

Training an RNN Language Model

- Use text as a sequence of input-output pairs shifted by one word.
- For **hello world**, input hello predicts output world.
- Cross-entropy loss compares softmax output with the true next word.
- Back-propagation through time updates shared recurrent weights.

RNNs for Token-Level Classification

- Each timestep can output a label for the current word.
- POS tagging predicts tags; coreference predicts antecedents.
- Training requires labeled sequences, not just raw text.
- Bidirectional RNNs use both left and right context.

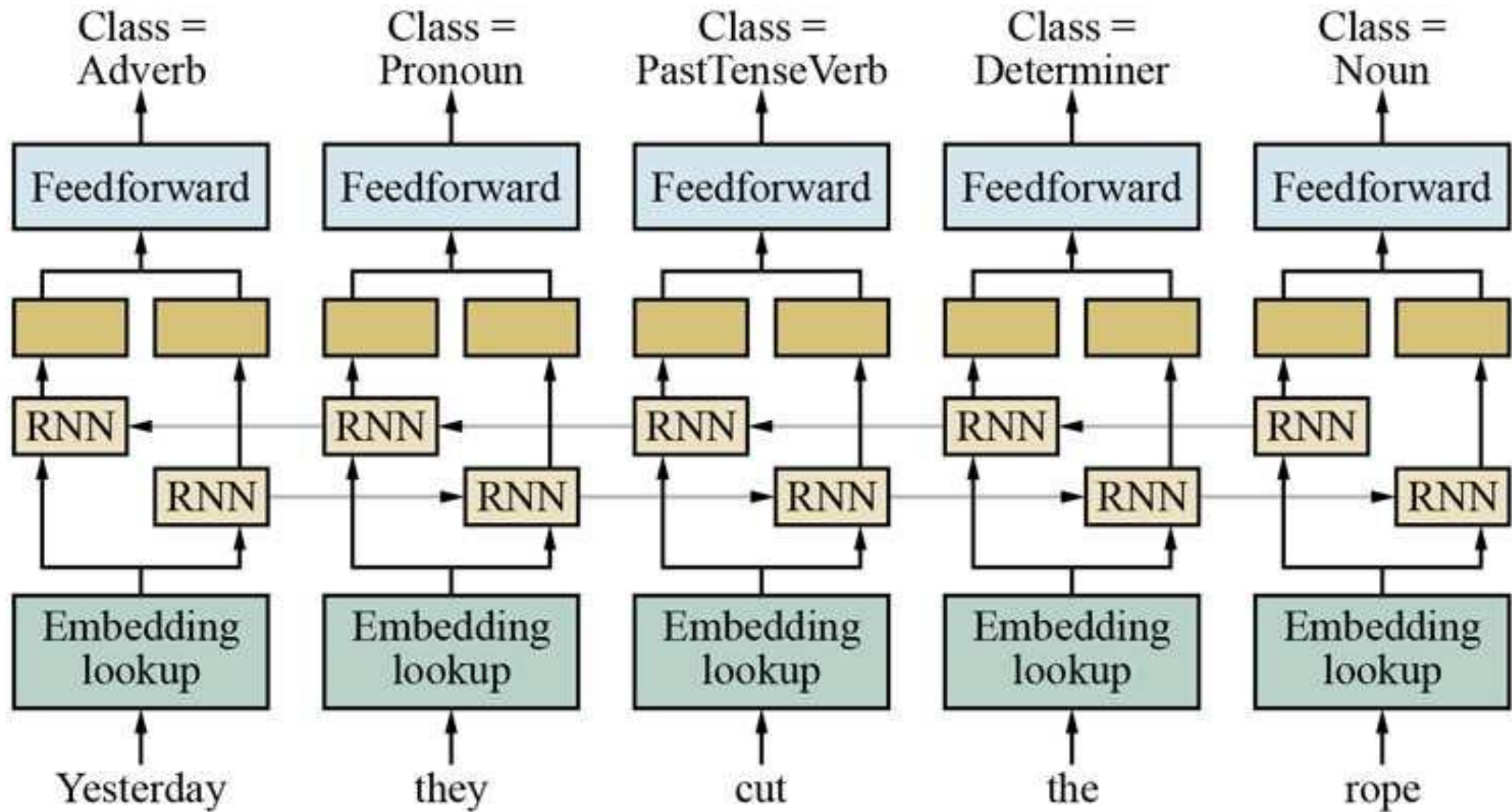


Figure 24.5. A bidirectional RNN network for POS tagging.

Sentence-Level Classification

- Some tasks need one label for the whole sentence or document.
- Examples: **sentiment**, topic, intent, spam detection.
- The final hidden state can summarize the input sequence.
- Pooling over hidden states is another common strategy.

LSTMs and the Long-Context Problem

- Plain RNNs can lose or distort information over many steps.
- This resembles the **vanishing gradient** problem.
- LSTMs add gates to control what to remember or forget.
- They improved many NLP tasks before transformers became dominant.

RNN Strengths and Weaknesses

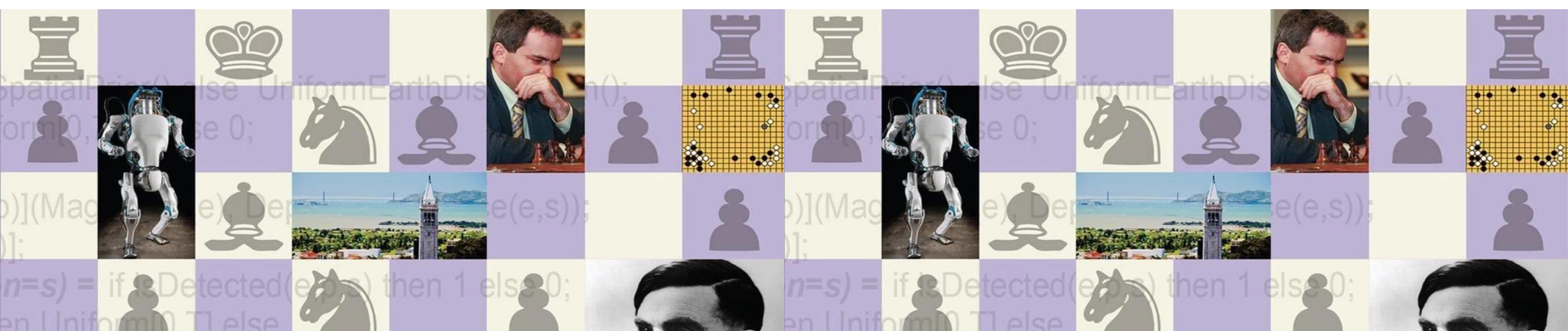
■ Strength:

- Naturally processes variable-length sequences.
- Hidden states adapt to task-relevant information.

■ Weakness:

- Sequential computation limits parallelism.
- Very long dependencies remain difficult.

Sequence-to-Sequence Models



Machine Translation as Sequence Transduction

- Input and output are both sequences, often with different lengths.
- The target word depends on the full source sentence and prior target words.
- Training uses aligned source-target sentence pairs.
- The goal is fluent text that preserves meaning.

Encoder-Decoder Architecture

- The **encoder** reads the source sequence.
- The **decoder** generates the target sequence.
- A hidden representation connects source understanding to target generation.
- Generation stops when the model emits an end-of-sequence token.

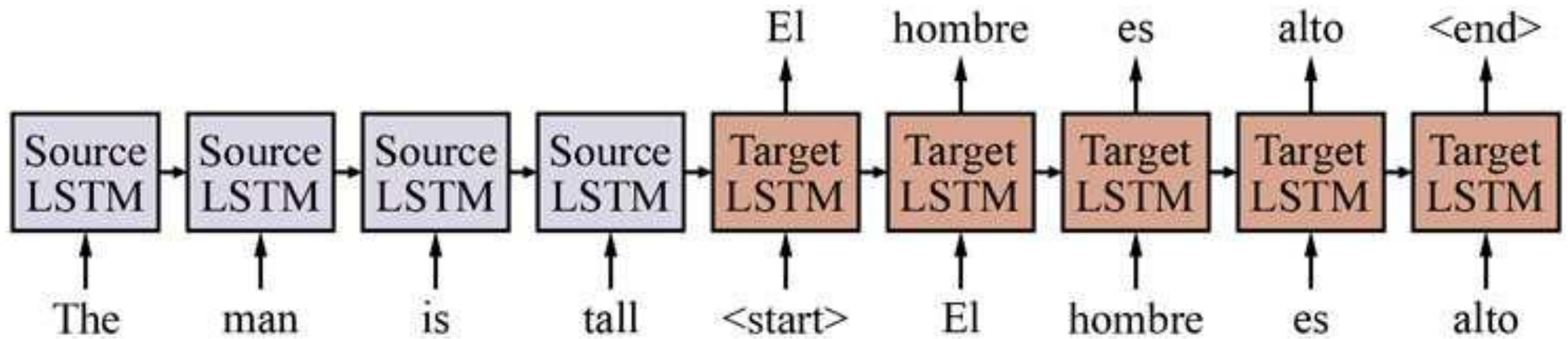


Figure 24.6. A basic sequence-to-sequence model for translation.

Three Shortcomings of Basic Seq2Seq

- **Nearby context bias:** hidden state favors recent words.
- **Limited context:** source sentence is compressed too much.
- **Expensive search:** target sequences create many hypotheses.
- These limitations motivate attention and better decoding.

Attention: The Core Idea

- The decoder should focus on different source words at each step.
- Attention assigns a **soft weight** to each source position.
- A context vector is the weighted sum of source hidden states.
- The mechanism is differentiable and trainable end-to-end.

Attention Equations

- Score: $r_{ij} = h_{i-1} \cdot s_j$
- Normalize: $a_{ij} = \exp(r_{ij}) / \sum_k \exp(r_{ik})$
- Context: $c_i = \sum_j a_{ij} s_j$
- Softmax turns raw scores into a probability distribution.

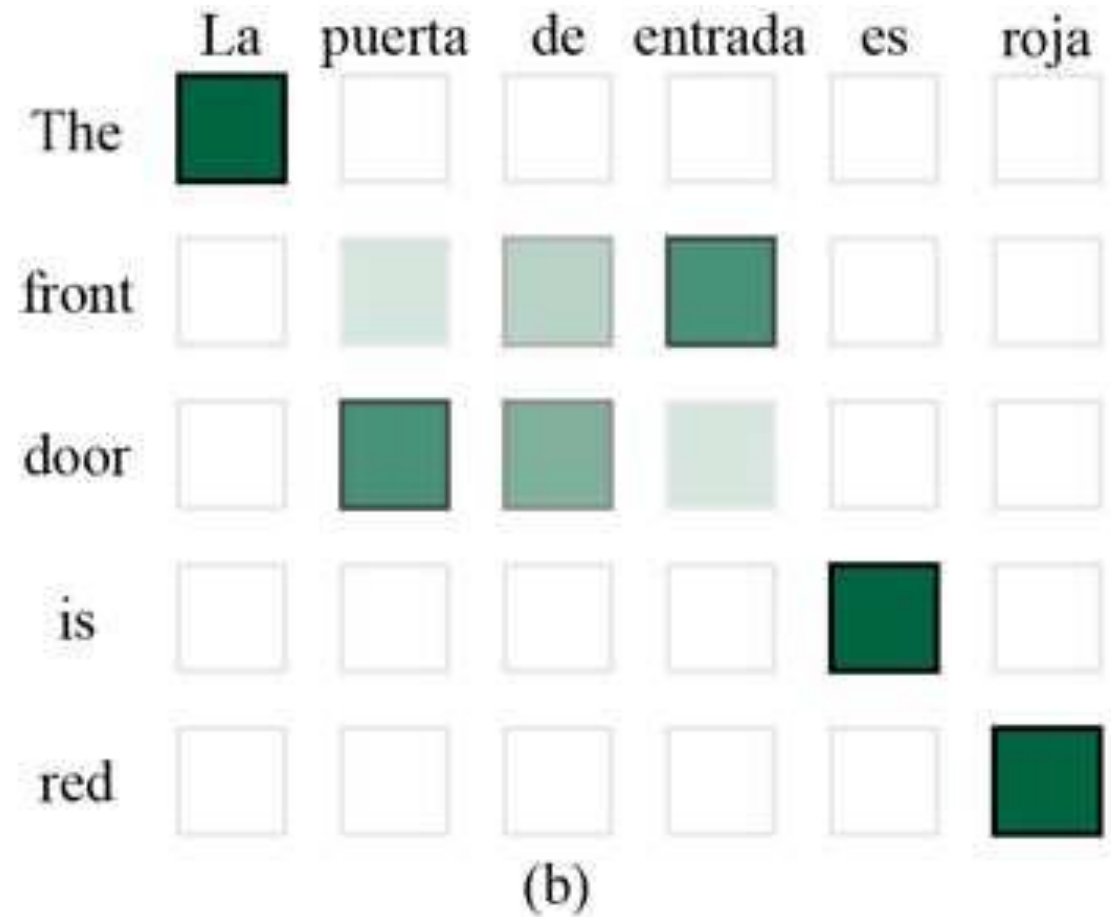
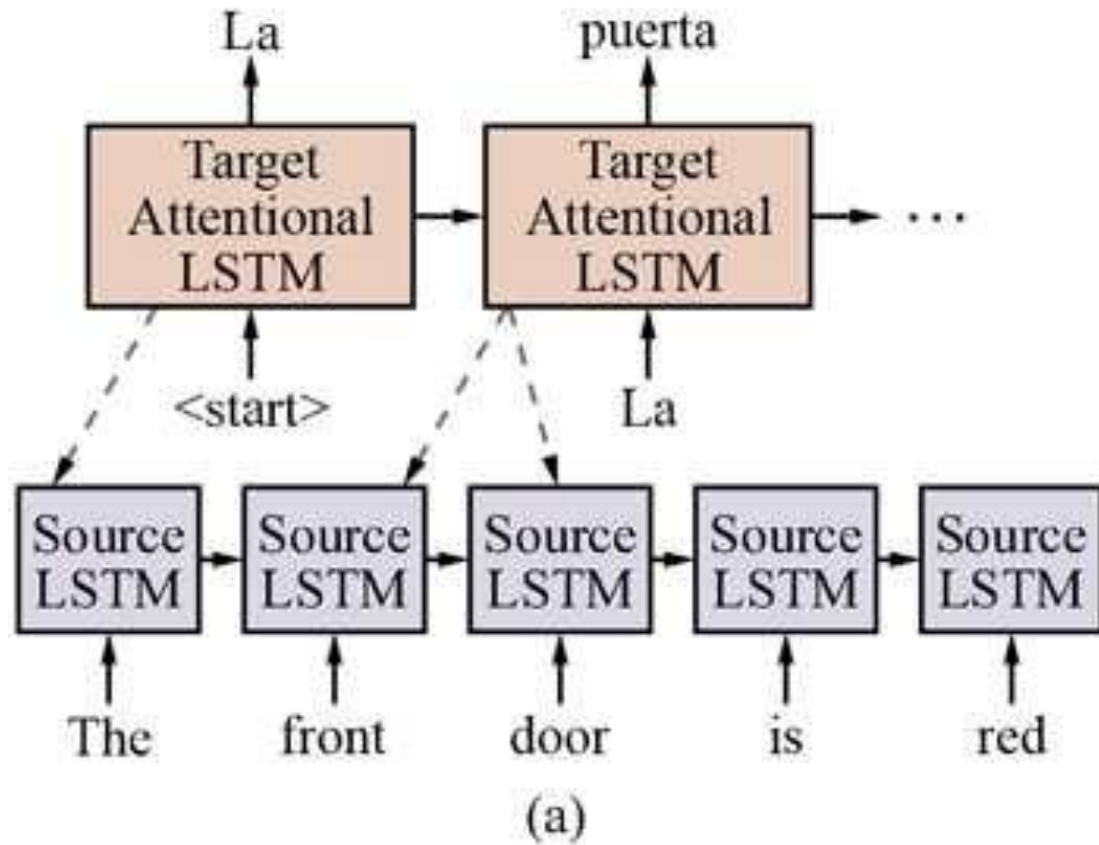


Figure 24.7. Attention links target words to relevant source words.

Decoding: Greedy vs Beam Search

- **Greedy decoding** chooses the best next word at each step.
- It may miss the best whole sentence.
- **Beam search** keeps the top b partial hypotheses.
- Hypothesis score is the sum of log word probabilities.



Figure 24.8. Beam search keeps multiple translation hypotheses.

Beam Size Tradeoff

- Small beams are faster but risk missing good translations.
- Large beams explore more hypotheses but cost more computation.
- Very large beams may favor overly short or generic outputs.
- Decoding is a search problem over possible target sequences.

The Transformer Architecture



Why Transformers?

- RNNs process sequences one step at a time.
- Transformers use **self-attention** to connect all positions directly.
- They model long-distance context without sequential dependency.
- Matrix operations make them efficient on GPUs and TPUs.

Self-Attention

- Each word attends to other words in the same sequence.
- Self-attention creates a context-aware vector for each position.
- The relation is **asymmetric**: attention from i to j can differ from j to i .
- All positions can be processed in parallel.

Queries, Keys, and Values

- Each input vector is projected into **query**, **key**, and **value** vectors.
- Query asks: what am I looking for?
- Key answers: what information do I contain?
- Value provides the content to be combined.

Scaled Dot-Product Attention

- Raw score: dot product between query and key.
- Scale by $\sqrt{d}$ for numerical stability.
- Softmax converts scores into attention weights.
- Output is the weighted sum of value vectors.

From Self-Attention to Transformer Layers

- A transformer layer includes self-attention and a feedforward network.
- **Residual connections** help gradients flow through deep stacks.
- Layer outputs become inputs to the next layer.
- Practical models usually stack many layers.

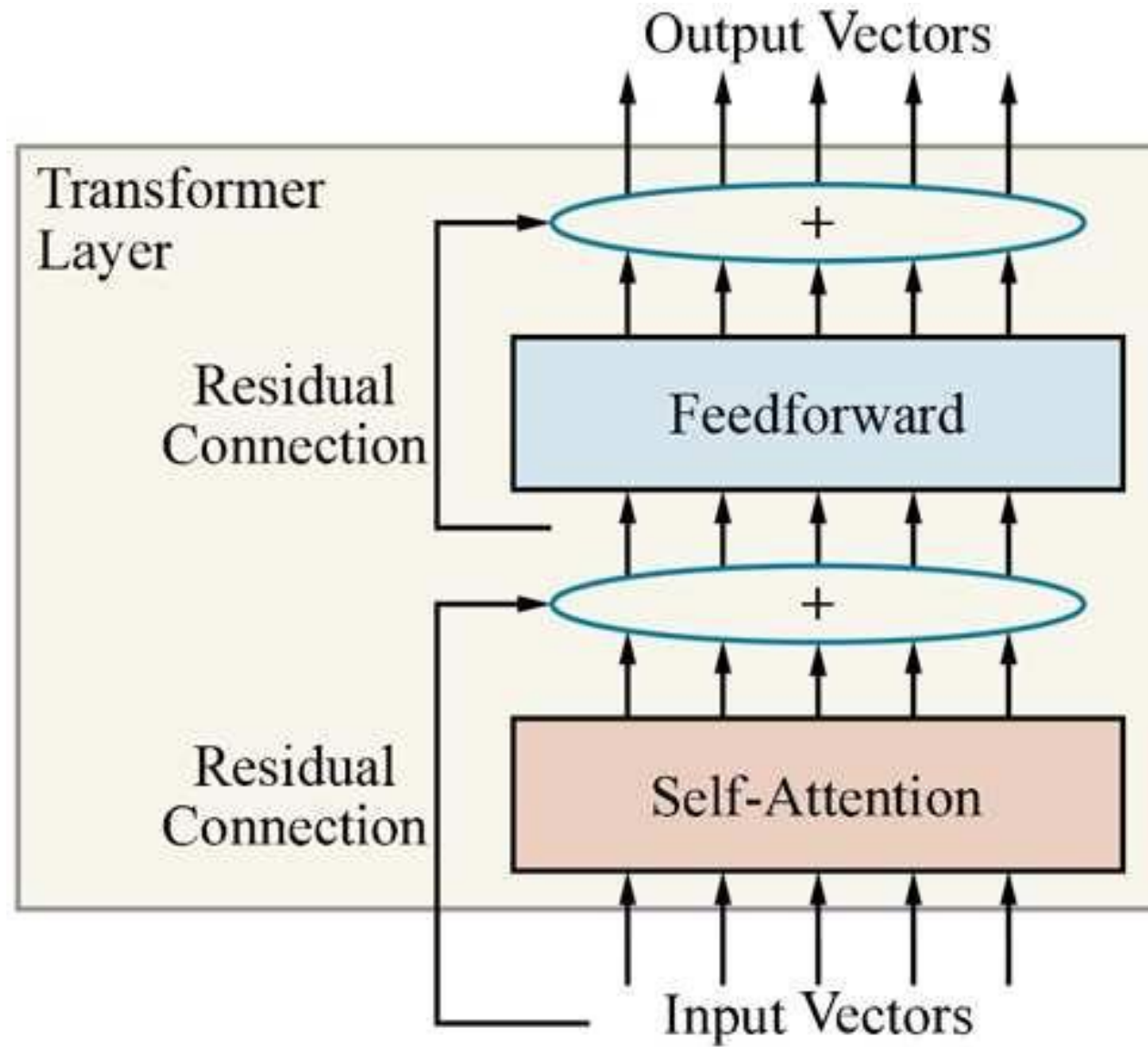


Figure 24.9. A single-layer transformer with self-attention and feedforward sublayers.

Positional Embeddings

- Self-attention alone is insensitive to word order.
- Transformers add **positional embeddings** to word embeddings.
- The input vector represents both word identity and position.
- Order information becomes available to every layer.

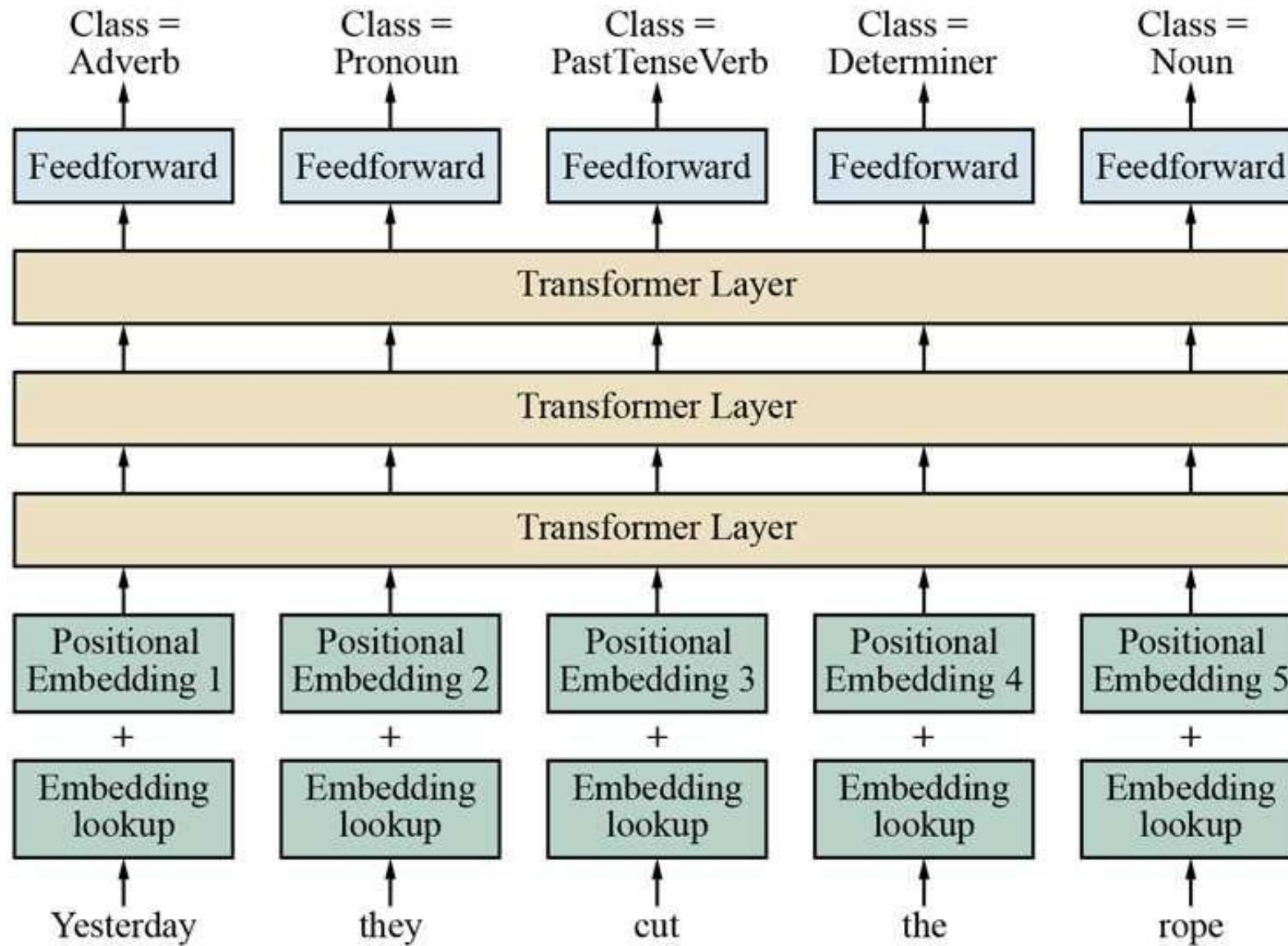


Figure 24.10. Transformer architecture for POS tagging.

Transformer Encoder and Decoder

- The chapter first describes the **transformer encoder**.
- Encoders are useful for classification and tagging tasks.
- Full machine translation also uses a **transformer decoder**.
- The decoder attends to encoder outputs while generating target text.

Transformers vs RNNs

- RNNs encode order naturally but compute sequentially.
- Transformers need position information but compute in parallel.
- Self-attention gives shorter paths between distant words.
- The cost grows with sequence length because all pairs are considered.

Pretraining and Transfer Learning



Why Pretraining Fits NLP

- Labeled linguistic data is expensive to create.
- Unlabeled text is abundant on the Internet.
- Pretraining learns general linguistic knowledge from raw text.
- Fine-tuning adapts the model to a specific task.

Pretrained Word Embeddings

- WORD2VEC and GloVe learn vectors from large unlabeled corpora.
- GloVe uses word **co-occurrence counts** within a context window.
- Dot products approximate log co-occurrence probabilities.
- Pretrained vectors can be reused in downstream models.

Domain-Specific Embeddings

- Embeddings can encode knowledge from specialized corpora.
- Scientific, medical, or legal text contains domain-specific relations.
- A general embedding may miss these relations.
- Training on the right corpus can improve downstream performance.

Contextual Representations

- A contextual vector depends on both the word and its surrounding context.
- The same word can receive different vectors in different sentences.
- This addresses **polysemy** and some idiomatic usage.
- Contextual models became central to modern transfer learning.

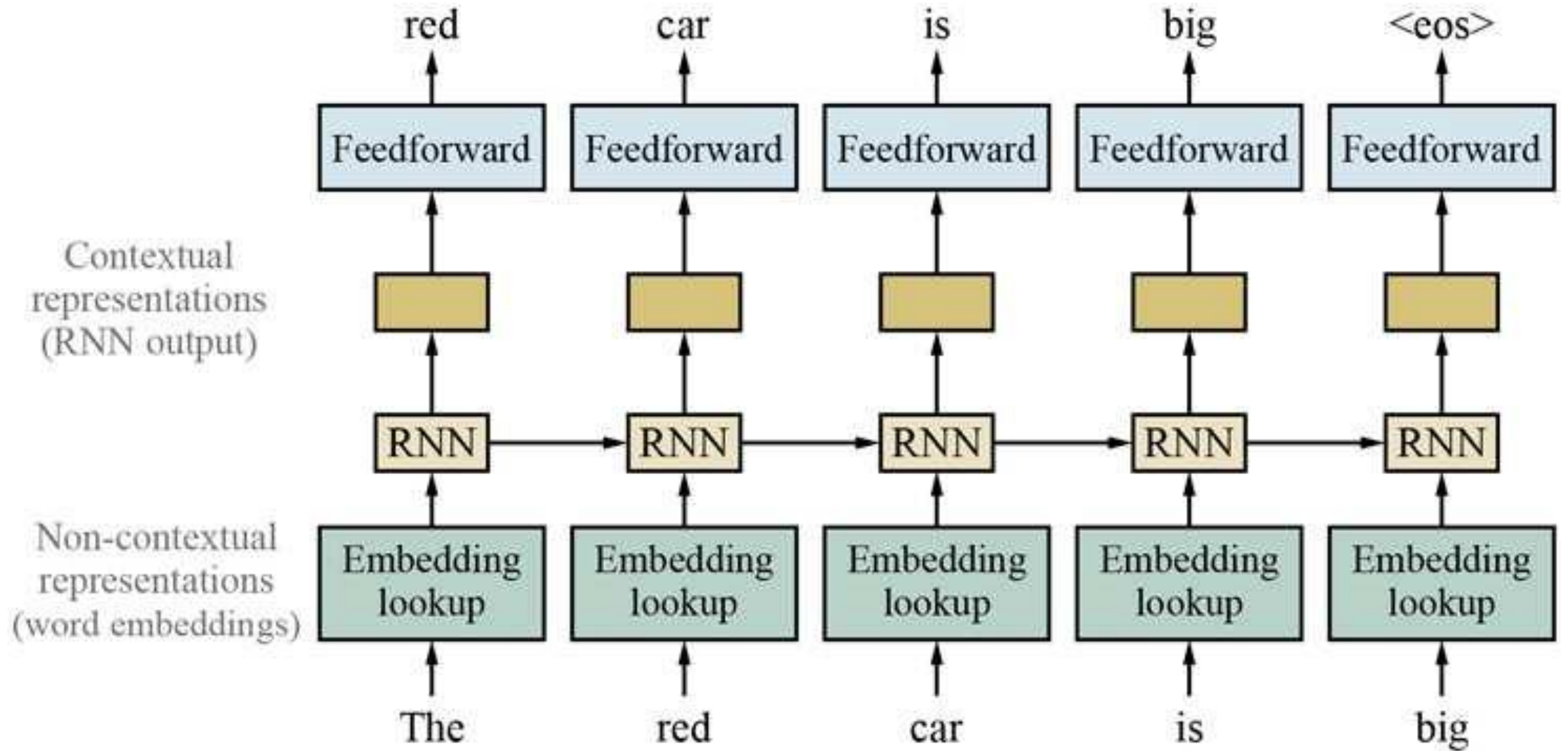


Figure 24.11. Training contextual representations with a left-to-right language model.

Masked Language Models

- Mask some input words and train the model to recover them.
- The sentence itself supplies the training label.
- Bidirectional context can be used around the mask.
- After pretraining, the model can be fine-tuned for many tasks.

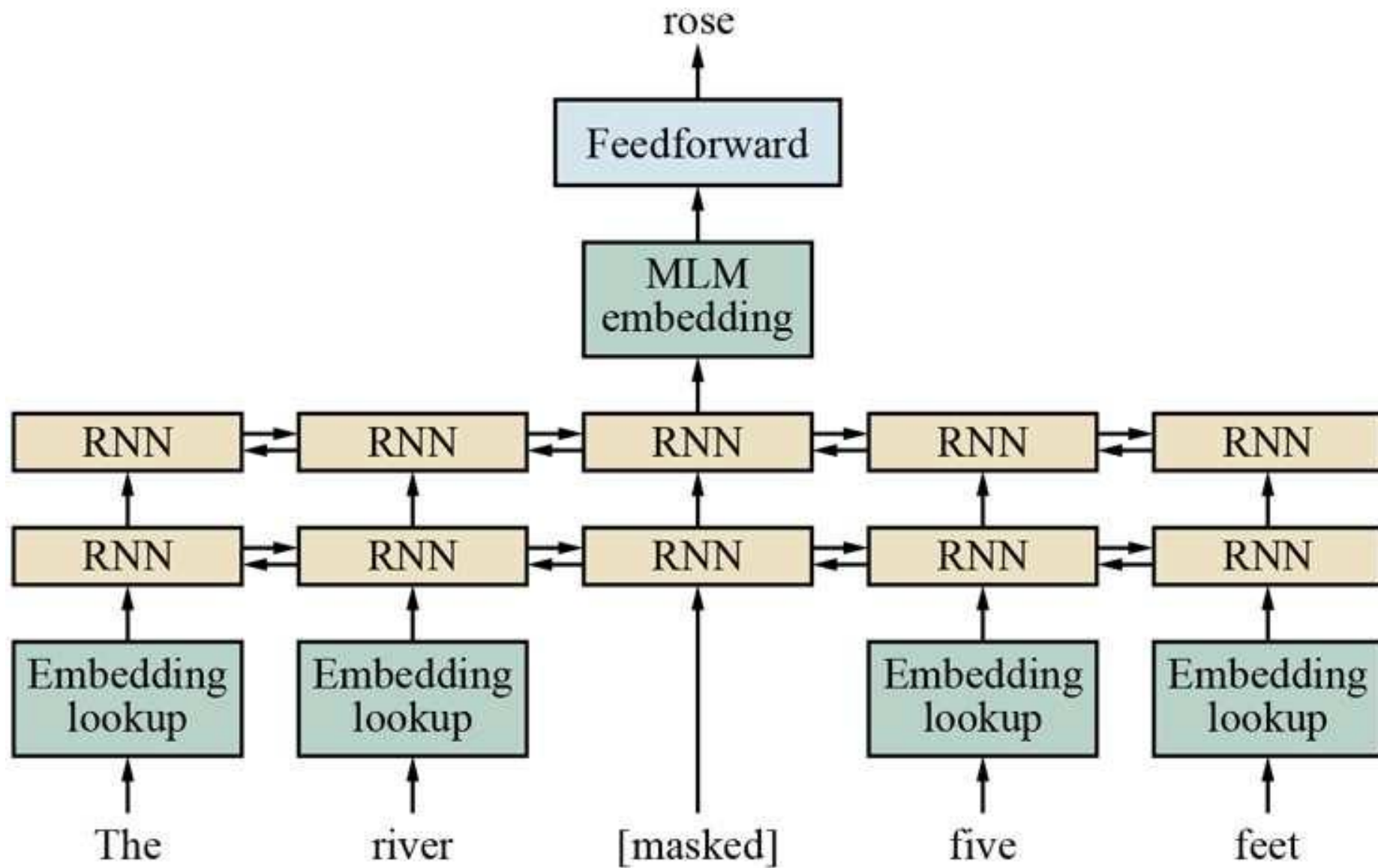


Figure 24.12. Masked language modeling predicts masked input words.

Fine-Tuning and Transfer

- Start with a model pretrained on massive unlabeled text.
- Add or adapt an output layer for the target task.
- Train further on a smaller labeled dataset.
- The same pretrained model can support QA, entailment, tagging, and classification.

State of the art



NLP's ImageNet Moment

- Around 2018, transfer learning produced rapid NLP progress.
- Pretrained models became reusable foundations for many tasks.
- Hardware advances made very large models feasible.
- Downloading a pretrained model became more practical than training from scratch.

Milestones in Modern NLP

- **WORD2VEC** and **GloVe** popularized reusable word vectors.
- **ELMo** introduced deep contextual word representations.
- **BERT** used masked language modeling for bidirectional pretraining.
- **GPT-style models** showed strong left-to-right generation.

ARISTO and Question Answering

- ARISTO answered 8th-grade science multiple-choice questions.
- It used an **ensemble** of solvers, including large language models.
- ROBERTA alone achieved a strong score on the exam.
- The task requires language, commonsense, and science knowledge.

1. **What will best separate a mixture of iron filings and black pepper?**
(a) magnet (b) filter paper (c) triple beam balance (d) voltmeter
2. **Which form of energy is produced when a rubber band vibrates?**
(a) chemical (b) light (c) electrical (d) sound
3. **Because copper is a metal, it is**
(a) liquid at room temperature (b) nonreactive with other substances
(c) a poor conductor of electricity (d) a good conductor of heat
4. **Which process in an apple tree primarily results from cell division?**
(a) growth (b) photosynthesis (c) gas exchange (d) waste removal

Figure 24.13. ARISTO answers 8th-grade science questions.

Current Limitations

- Large models require enormous text and compute.
- Transformer context is often limited to a few hundred or thousand tokens.
- Models can produce fluent but incorrect or unstable text.
- They still lag humans on many forms of robust understanding.

Toward Hybrid Approaches

- Data-driven models are easier to build and maintain today.
- They may learn latent structure similar to grammar and semantics.
- But we do not fully understand what large models represent.
- Future systems may combine neural learning with explicit structure.

Summary

- **Word embeddings** replace atomic symbols with robust continuous vectors.
- **RNNs and seq2seq models** handle ordered text and generation.
- **Attention and transformers** model long-distance context efficiently.
- **Pretraining and transfer learning** turn unlabeled text into reusable knowledge.
- **State-of-the-art NLP** is powerful, but still limited and data-hungry.

